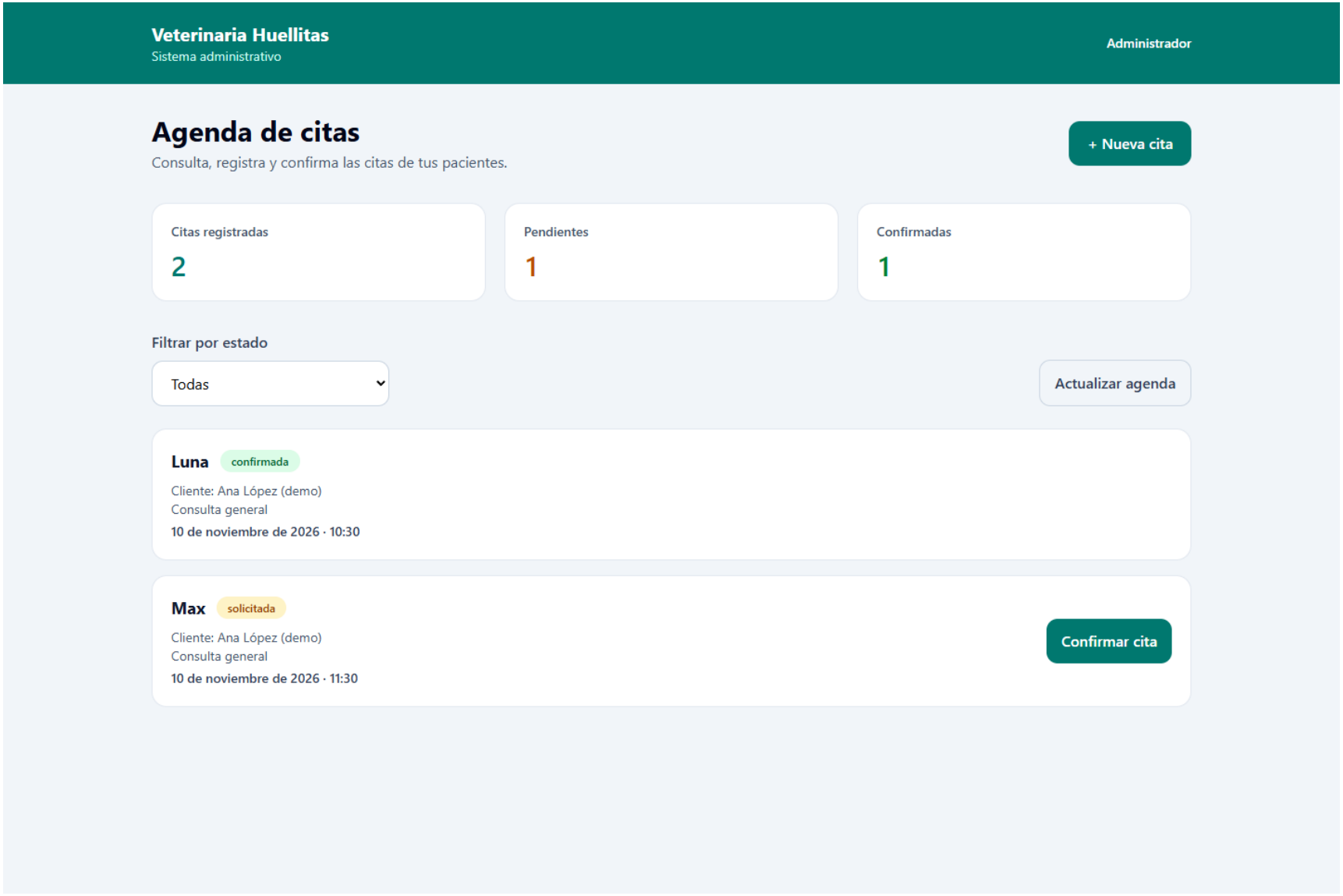
Actividad Sumativa 1 - Aplicación funcional

Alan Daniel de la Cruz Loeza | Aplicaciones Web | Unidad 3 | 9 de noviembre de 2026

Se comprobó la consulta, el registro mediante **POST /api/citas** y la confirmación mediante **PATCH**. React consume el API de Express y las citas se guardan en SQLite, una base relacional en disco. Los nombres y pacientes mostrados son datos ficticios de demostración.

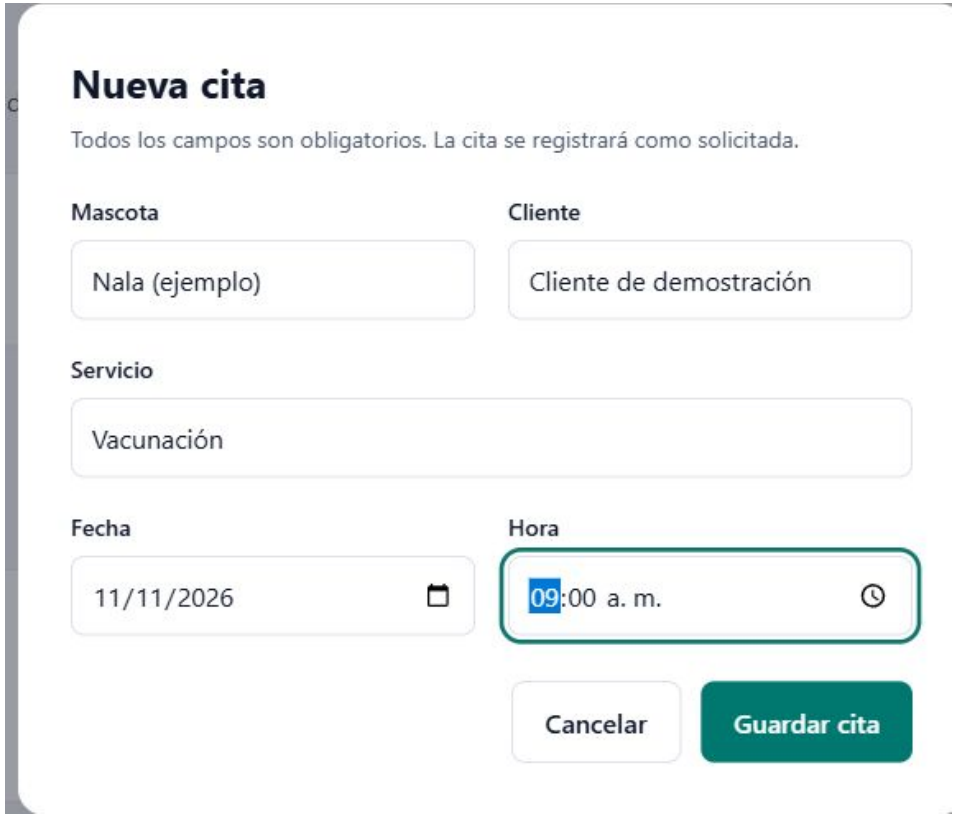


Registro y validaciones visibles

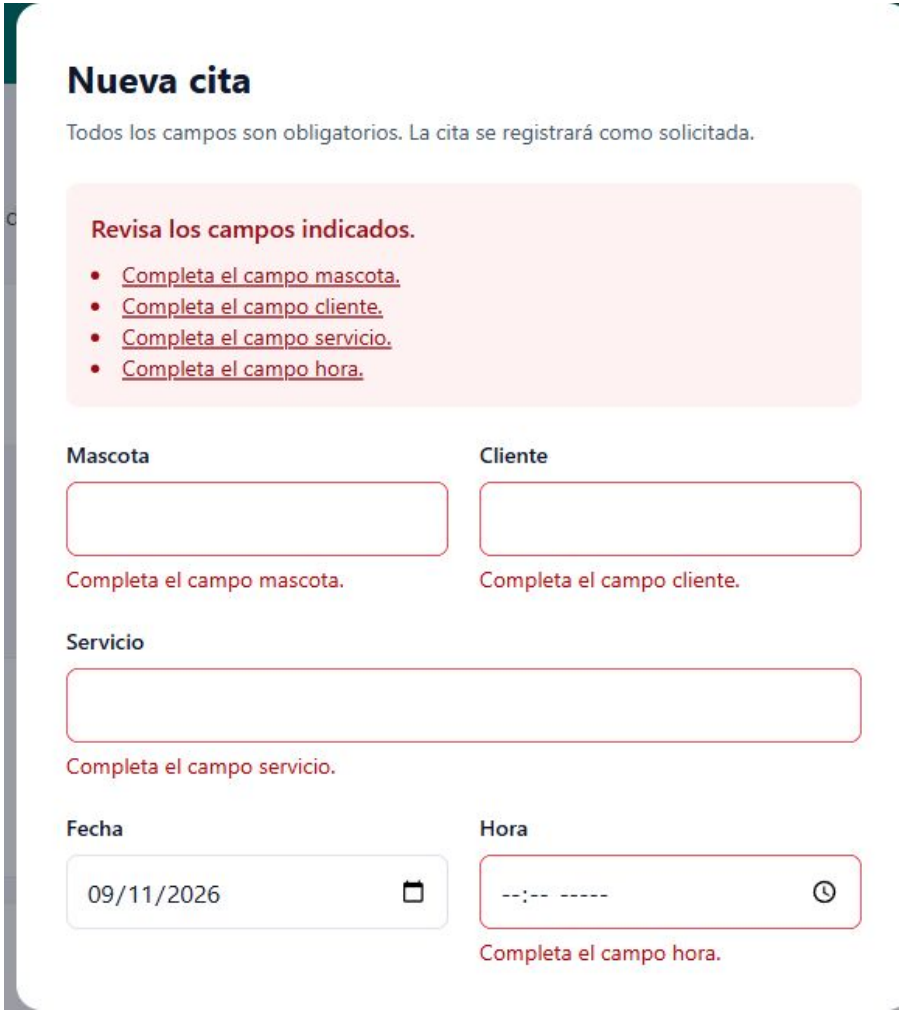
Alan Daniel de la Cruz Loeza | Aplicaciones Web | Unidad 3 | 9 de noviembre de 2026

El formulario solicita mascota, cliente, servicio, fecha y hora. Los errores se muestran junto al campo y en un resumen. El backend vuelve a validar y responde **409** si otra cita activa ocupa el horario.

Formulario con datos de ejemplo



Validación de campos obligatorios



También se verificó el aviso de horario ocupado y la recuperación tras una interrupción de red simulada. El formulario conserva lo escrito cuando el API rechaza la solicitud.

Persistencia y pruebas realizadas

Alan Daniel de la Cruz Loeza | Aplicaciones Web | Unidad 3 | 9 de noviembre de 2026

Prueba de persistencia: se consultó la agenda, se cerró el proceso del backend y se inició otra vez desde la carpeta final. La respuesta anterior y la posterior coincidieron, incluidos los identificadores y el estado confirmado. SQLite devolvió **ok** en su comprobación de integridad.

ID	Mascota	Fecha	Hora	Estado al volver a iniciar
3	Luna	2026-11-10	10:30	confirmada
4	Max	2026-11-10	11:30	solicitada

Verificación automática: 16 pruebas del API aprobadas, sin fallos. Cubren GET, POST, PATCH, PUT, DELETE, validaciones, JSON mal formado, horarios repetidos, dos solicitudes simultáneas y reinicio del backend. La compilación de frontend y backend y la revisión con ESLint finalizaron correctamente.

Verificación en navegador: alta, confirmación, filtros, recarga, errores y reintento. Se revisaron anchos de 1440, 768 y 390 px, sin desbordamiento horizontal. El diálogo se cierra con Escape y devuelve el foco al botón que lo abrió.

Cómo repetir: ejecutar `npm run lint`, `npm run build:all` y `npm test`. Las pruebas crean una base temporal distinta de la agenda de demostración.

Repositorio de entrega:
https://github.com/alannloezaa/appVeterinaria-U3-Alan

Flujo que se debe explicar: formulario, fetch, Express, ruta, controlador, repositorio, base de datos, respuesta y React. La guía incluida desarrolla el recorrido y el manejo de errores con `asyncHandler`.